Producto final sobre herencia en JPA

Puzzle de Aronson · AD y PSP UD2 · Hoja 1 de 2

Nombre: _____ Grupo base: _____

Completa esta comparativa mientras escuchas a los demás expertos. Después, acordad la decisión de diseño de la segunda hoja. Cada integrante debe poder explicar el resultado.

1 Comparamos las cuatro opciones

Modelo común: Contenido abstracta (id, titulo), Pelicula (duracionMinutos) y Serie (numeroTemporadas). Escribe respuestas breves con tus propias palabras.

Opción	Tablas necesarias	Dónde se guardan id y titulo	¿Se puede consultar Contenido como entidad? ¿Cómo?
A · SINGLE_TABLE	Una tabla: contenido	id, titulo y los campos de Pelicula y Serie en contenido	Sí. Contenido es @Entity; findAll recupera ambos subtipos.
B · JOINED	contenido, pelicula y serie	id y titulo en contenido; cada subtipo guarda sus campos propios	Sí. Contenido es @Entity; Hibernate une las tablas para reconstruirlos.
C · TABLE_PER_CLASS	Una tabla por clase concreta: pelicula y serie	id y titulo se repiten en cada tabla concreta	Sí. La consulta a Contenido combina las tablas concretas (UNION).
D · @MappedSuperclass	Una tabla por entidad concreta; ninguna para la base	id y titulo se heredan del mapeo, pero se guardan en cada tabla concreta	No. Contenido no es entidad; no hay consulta polimórfica desde esa base.

2 Valoramos ventajas y limitaciones

Opción	Una ventaja o uso adecuado	Una limitación o coste
A · SINGLE_TABLE	Consulta conjunta sencilla; adecuada con pocos campos específicos.	Muchas columnas NULL y discriminador; campos específicos no siempre NOT NULL.
B · JOINED	No duplica campos comunes; admite restricciones en cada subtipo.	Recuperar tipos completos requiere JOIN y puede encarecer consultas.
C · TABLE_PER_CLASS	Lectura directa de cada tipo; sin JOIN entre tablas para recuperarlo.	Duplica campos comunes; consulta conjunta requiere UNION.
D · @MappedSuperclass	Reutiliza el mapeo común sin crear tabla ni entidad base.	No permite repositorio, consulta ni relaciones polimórficas por Contenido.

Pregunta: ¿qué diferencia a @MappedSuperclass de las tres estrategias de herencia entre entidades?

@MappedSuperclass solo aporta mapeo heredado: no es entidad ni tiene tabla propia. Las tres estrategias usan una entidad base y permiten consultas polimórficas.

Nuestra decisión de diseño

Puzzle de Aronson · AD y PSP UD2 · Hoja 2 de 2

Grupo base: _____ Integrantes: _____

3 Elegimos para una necesidad concreta

Identificad el encargo asignado para vuestro grupo.

A. Catálogo unificado. Se consulta continuamente una lista conjunta de películas y series y hay pocos atributos específicos.

B. Datos específicos obligatorios. Cada subtipo tiene numerosos atributos propios y se quieren imponer restricciones NOT NULL en sus tablas.

C. Reutilización técnica. Dos entidades comparten id y fechaCreacion, pero no se necesita consultarlas ni relacionarlas mediante una entidad base común.

Opción elegida y anotación principal:

Encargo A: catálogo unificado

Opción elegida: SINGLE_TABLE. Anotación: @Inheritance(strategy = InheritanceType.SINGLE_TABLE) en Contenido; Hibernate distingue el subtipo con un discriminador (por ejemplo, DTYPE).

Dos razones relacionadas con las necesidades del encargo:

1) Un catálogo se obtiene desde una sola tabla, sin combinar tablas. 2) Hay pocos atributos específicos, así que los NULL de los campos no aplicables son un coste asumible.

Inconveniente que aceptamos y alternativa que hemos considerado:

Coste: tabla ancha, con campos específicos NULL para el otro tipo; no se pueden exigir fácilmente como NOT NULL. Alternativa considerada: JOINED, si fueran importantes las restricciones por subtipo.

4 Representamos la solución

Dibujad las tablas y sus columnas. Marcad PK y FK, si procede. En A o B, situad Órbita (id 1, 120 minutos) y Horizonte (id 2, 3 temporadas). En C, elegid dos entidades concretas e indicad dónde aparecen id y fechaCreacion.

CONTENIDO (id PK, DTYPE, titulo, duracion_minutos, numero_temporadas)

1 | Pelicula | Órbita | 120 | NULL

2 | Serie | Horizonte | NULL | 3

¿Qué repositorio o repositorios usaríamos y cómo consultaríamos los datos?

Usaríamos JpaRepository<Contenido, Long>. findAll() devuelve películas y series en el mismo catálogo; el discriminador permite a Hibernate crear cada objeto con su subtipo correspondiente.